AquaMark — Comprehensive Academic & Technical Learning Guide

Official Academic Title: Robust Digital Watermarking for Protection of Electronic Course Content

Student Learning Handbook, Code Audit, Implementation Breakdown, and Viva Preparation Guide

---

★ How to Use This Handbook:
This guide is written especially for you as a student. It starts from fundamental concepts, explains every single technical term when introduced, walks through the exact implemented Python and React code, clarifies honest security limits, and provides complete viva answers so you can defend your project with absolute confidence.

1. Honest Implementation Audit (What is Built vs What is Planned)

In any senior or capstone engineering evaluation, academic integrity is paramount. Never claim a feature exists if it was only a design proposal. Below is the audited status of the AquaMark codebase:

A. Features Fully Implemented and Verified in Working Code

- **1. Visible Vector Watermark Engine:** Implemented in backend/app/watermarking/visible.py using PyMuPDF (fitz). Preserves PDF page geometry, injects diagonal text overlays with adjustable opacity/angles, and embeds top security header banners.
- **2. 2D-DCT Invisible Watermark Pipeline:** Implemented in backend/app/watermarking/invisible_dct.py. Converts PDF page to Luminance (Y), divides into 8x8 blocks, embeds 128-bit payload with preamble sync marker [1,0,1,0,1,1,0,0] into mid-frequency coefficients (3,2) and (2,3), and performs inverse DCT.
- **3. Ed25519 Asymmetric Digital Signatures:** Implemented in backend/app/cryptography/engine.py using cryptography library. Generates Ed25519 keypairs, serializes canonical JSON manifests, and verifies digital signatures.
- **4. File Integrity & HMAC:** SHA-256 block-based hashing for original and protected files; HMAC-SHA-256 binding original hash to protected hash.
- **5. Authenticated Symmetric Encryption:** AES-256-GCM encryption with unique 96-bit random nonces for sensitive metadata.
- **6. Tamper-Evident Audit Hash Chain:** Implemented in backend/app/services/audit.py. Each event's SHA-256 hash chains to the previous event's hash, forming an append-only cryptographic sequence.
- **7. Robustness Testing Lab:** Implemented in backend/app/watermarking/robustness.py & frontend/src/pages/Lab.tsx. Evaluates watermark recovery under JPEG compression, geometric scaling, Gaussian noise, and cropping, calculating PSNR (dB) and Bit Error Rate (BER).
- **8. Complete Full-Stack REST API & React Frontend:** FastAPI endpoints under /api/v1 (auth, documents, watermarks, verify, experiments, dashboard); React 19 + TypeScript + Tailwind CSS UI.
- **9. Automated Test Suite:** 8 automated test cases in backend/tests/ passing at 100% (SHA-256, HMAC, Ed25519, AES-GCM, DCT pipeline, health, auth, access prevention).

B. Features Implemented But Not Separately Unit Tested

- **1. Multi-page Batch Invisible Watermarking:** Embedding payload into carrier page 1 is tested; multi-page distributed spreading is supported in code but unit tests target page 1 carrier.
- **2. Contrast Adjustment Transformation:** Backend supports contrast_adjust in run_single_experiment; the four primary tested vectors in the frontend UI are JPEG, resize, noise, and crop.

C. Features Planned as Future Work (Not Implemented)

- **1. Hardware Token / HSM Integration:** Storing private Ed25519 keys on physical YubiKeys or cloud HSMs (AWS KMS / GCP Cloud KMS). Currently, keys are stored on the server filesystem.
- **2. Automatic Projective Mesh Rectification:** Self-healing detection of severe trapezoidal phone camera photos taken of computer screens.
- **3. Outbound Email Delivery:** SMTP / SendGrid email verification. Currently, user accounts activate directly upon registration.

D. Security Non-Claims (Honest Academic Boundaries)

- **No 'Copy-Proof' Claim:** Digital watermarking does NOT physically lock down computer displays. If a student can read the document, they can take a screenshot or retype it. Watermarking is for provenance and leak tracing.
- **No 'Proof of Legal Ownership' Claim:** An Ed25519 signature proves that a specific private key signed the manifest on that date; it does NOT legally prove who first created the underlying intellectual content.
- **No Universal Robustness:** Extreme cropping (>30% of page area) or severe geometric rotation disrupts 8x8 block alignment and prevents payload recovery.

2. Project Overview & The Real-World Story

Scenario: Dr. Katherine's Leaked Midterm:
Dr. Katherine teaches Advanced Cryptography (CS401) to 60 university students. On Monday, she uploads the upcoming Midterm Exam Assignment to the university portal. On Tuesday morning, a teaching assistant notices that the exact assignment is posted on Chegg and a public Telegram group.
Dr. Katherine faces three questions: (1) Is this file actually her authentic exam paper, or a modified version where someone altered Question 4? (2) Can she prove to the Department Chair that she published this document first, without exposing her master private keys? (3) Which of her 60 students downloaded this specific copy and leaked it?
Traditional tools fail: cosmetic watermarks are cropped in seconds, and SHA-256 breaks if the student saves as a JPEG. AquaMark solves this by combining visible watermarks, invisible frequency steganography, and Ed25519 digital signatures.

3. Basic Concepts Explained from Scratch (No Jargon)

- **SHA-256 Hash:** A mathematical fingerprint. Fixed 64 hex characters. Deterministic (same file = same hash) and has the Avalanche Effect (change 1 comma = completely different hash).
- **HMAC (Hash-based Message Authentication Code):** A hash mixed with a secret key. Proves that data was not altered AND was approved by the secret key holder.
- **Asymmetric Digital Signature (Ed25519):** A keypair system. Private key stays on server to sign manifests; public key is shared to verify signatures without anyone being able to forge them.
- **Digital Watermark:** Information embedded directly into the carrier media pixels. Visible watermarks deter casual leaks; invisible DCT watermarks survive compression and formatting.
- **Hash Mismatch vs Malicious Tampering:** A hash mismatch proves binary bytes changed, but does NOT prove malice (e.g. re-saving in a different PDF reader alters metadata). AquaMark verifies if the underlying DCT watermark payload is still intact.

4. System Architecture & Technology Stack

5. Detailed Breakdown of Every Implemented Module

Module 1: Authentication & Account Security

Genuine registration, login, and JWT session authentication. Passwords hashed using Argon2id with memory-hard parameters. First user is 'admin', others 'instructor'. Files: backend/app/api/v1/auth.py, passlib[argon2], pyjwt. Limitation: No outbound SMTP confirmation emails. Viva Tip: Argon2id prevents GPU brute-forcing.

Module 2: Secure PDF Ingestion & Vault Storage

Validates PDF extension, limits page count (1-100) and size (25MB) using PyMuPDF. Generates random UUIDs for storage outside web root to prevent path traversal. Calculates original SHA-256 hash. Files: backend/app/api/v1/documents.py. Limitation: No ClamAV antivirus daemon in local dev. Viva Tip: UUID naming prevents directory traversal attacks.

Module 3: Visible Digital Watermarking

PyMuPDF vector overlay applying 45-degree diagonal text with 35% alpha transparency and top security banner. Preserves existing page dimensions and underlying text searchability. Files: backend/app/watermarking/visible.py. Limitation: Can be cropped if margins are cut. Viva Tip: Preserves vector text without raster flattening.

Module 4: Invisible 2D-DCT Steganography Pipeline

Converts page to Luminance (Y) channel, splits into 8x8 blocks, and applies 2D-DCT. Embeds 128-bit payload with preamble [1,0,1,0,1,1,0,0] into mid-frequency coefficients (3,2) and (2,3) using relative coefficient modulation. Files: backend/app/watermarking/invisible_dct.py, SciPy, NumPy. Limitation: Carrier page is rasterized at 150 DPI. Viva Tip: Mid-frequencies balance visual invisibility with compression resilience.

Module 5: Cryptographic Manifest & Ed25519 Signatures

Constructs canonical JSON manifest (sorted keys, compact separators) with original/protected hashes and watermark metadata. Signed using Ed25519 private key. Public key stored in PEM. Files: backend/app/cryptography/engine.py. Limitation: Proves manifest integrity, not legal ownership of original text. Viva Tip: Canonicalization ensures identical byte strings before hashing.

Module 6: Forensic Verification Center

Multi-path forensic inspection: tests candidate SHA-256 against DB, validates Ed25519 signature on manifest, and extracts DCT frequency watermark. Issues official PDF certificates via ReportLab. Files: backend/app/api/v1/verify.py, pdf_generator.py. Limitation: Detached verification requires companion manifest JSON. Viva Tip: Distinguishes exact hash matches from transformed authentic files.

Module 7: Robustness Testing Lab

Interactive sandbox testing watermark survival against JPEG compression (Q10-Q95), downscaling/upscaling, additive Gaussian noise, and border cropping. Computes PSNR in dB and Bit Error Rate (BER). Files: backend/app/watermarking/robustness.py, frontend/src/pages/Lab.tsx. Limitation: Severe cropping (>20%) disrupts 8x8 block sync. Viva Tip: PSNR >35 dB indicates imperceptible distortion; 0% BER means perfect recovery.

Module 8: Tamper-Evident Chained Audit Logging

Chained ledger where each event's hash = SHA-256(Previous_Hash + Actor + Type + Status + Details). Displayed in real-time on dashboard. Files: backend/app/services/audit.py. Limitation: Root DB admin could recalculate chain without external public anchoring. Viva Tip: Hash chaining makes historical record deletion immediately detectable.

6. Testing Strategy & Actual Verified Results

All 8 automated test suites were executed using pytest and verified at 100% success:

Test Identifier	What It Verifies	Result
test_sha256_consistency	Determinism and avalanche effect on single byte alteration	**PASSED (100%)**
test_hmac_sha256_verification	HMAC verification with secret key & rejection of tampered data	**PASSED (100%)**
test_ed25519_digital_signature	Ed25519 signature creation & verification on canonical manifest	**PASSED (100%)**
test_authenticated_aes_gcm_encryption	AES-256-GCM encryption with 96-bit nonce & decryption	**PASSED (100%)**
test_dct_invisible_watermark_pipeline	Embedding 128-bit payload into 2D-DCT blocks & 100% extraction	**PASSED (100%)**
test_health	FastAPI server health endpoint responds with HTTP 200 HEALTHY	**PASSED (100%)**
test_auth_workflow_and_security	User registration, Argon2id hashing, login token & bad pass rejection	**PASSED (100%)**
test_unauthorized_access_prevented	Access to /documents without valid Bearer token returns HTTP 401	**PASSED (100%)**

7. Presentation Scripts for Any Time Limit

30-Second Elevator Pitch:

“My project is AquaMark, an applied cryptography platform that protects university lecture notes and exam papers from unauthorized distribution. We combine three layers: Visible Watermarks for visual deterrence, Invisible 2D-DCT Steganography that hides a recipient tracking ID inside pixel frequencies to survive JPEG compression and screenshots, and Ed25519 Digital Signatures that produce a tamper-evident cryptographic manifest. We also built an empirical Robustness Testing Lab that measures Peak Signal-to-Noise Ratio and Bit Error Rate across real-world distortion attacks.”

2-Minute Structured Explanation:

“In electronic education, high-value course content is frequently leaked. Traditional watermarks fail because any student can crop them out, while simple file hashes fail because recompressing a PDF alters the hash completely.
AquaMark implements three distinct mathematical layers: First, Visible Watermarking using PyMuPDF to place vector-preserving diagonal text across all pages. Second, Invisible DCT Steganography: we convert carrier pages to luminance, slice them into 8x8 blocks, and modulate mid-frequency transform coefficients (3,2) and (2,3) to embed a 128-bit

payload with preamble synchronization. This hidden serial number survives lossy JPEG compression and format conversions. Third, Cryptographic Authentication: we generate Ed25519 asymmetric digital signatures over canonical JSON manifests and log every action to a SHA-256 chained audit trail.
Finally, our platform includes a Robustness Lab where we empirically test and report watermark survival against JPEG compression, downscaling, noise, and cropping, outputting PSNR in decibels and Bit Error Rate percentages. All eight test suites are passing, and the application is live."

8. 25 Viva Examination Questions & Model Answers

Q1: What is the main objective of AquaMark?

To protect course content using visible watermarks for deterrence, invisible DCT steganography for leak tracing, and Ed25519 signatures for tamper-evident verification.

Q2: Spatial vs Frequency domain watermarking?

Spatial directly alters pixels (e.g. LSB), easily destroyed by JPEG. Frequency transforms image to coefficients (DCT) and embeds in mid-frequencies, surviving recompression.

Q3: Why 8x8 blocks for DCT?

Standard in JPEG compression, providing an ideal trade-off between localized frequency resolution and computation speed.

Q4: Which DCT coefficients are modified?

Mid-frequency pairs (3,2) and (2,3). Low frequencies contain visible shapes; high frequencies are discarded by JPEG. Mid-frequencies balance invisibility and robustness.

Q5: What is the preamble sync marker?

An 8-bit pattern [1,0,1,0,1,1,0,0] allowing the extractor to locate the payload start and verify signal presence before reading data.

Q6: Why Ed25519 instead of RSA-2048?

Provides 128-bit security with tiny 64-byte signatures, fast signing/verifying speeds, and immunity to timing attacks.

Q7: What is canonical JSON serialization?

Sorting keys and removing whitespace so the exact same byte string is generated every time before hashing and signing.

Q8: Why Argon2id for password hashing?

Memory-hard algorithm requiring configurable RAM and time, neutralizing GPU and ASIC brute-force attacks.

Q9: What is HMAC and why is it used?

Hash-based Message Authentication Code combining a secret key with file hashes, proving the pairing was authorized by the server.

Q10: What is AES-256-GCM?

Authenticated symmetric cipher encrypting sensitive metadata while generating an authentication tag with a 96-bit nonce.

Q11: Can a watermark prevent copying?

No. It is a forensic deterrence tool that allows tracing leakers after an unauthorized copy surfaces.

Q12: Does a signature prove legal copyright?

No. It proves the private key signed that manifest on that date; it does not prove original authorship.

Q13: What does PSNR stand for?

Peak Signal-to-Noise Ratio in dB. Values >35 dB indicate high visual fidelity indistinguishable from the original.

Q14: What is Bit Error Rate (BER)?

The fraction of recovered bits that differ from embedded bits; 0.0% means 100% accurate recovery.

Q15: How does the tamper-evident audit log work?

Each event hash = SHA-256(Previous_Hash + EventData). Altering any record invalidates all subsequent hashes.

Q16: Why store files with random UUIDs?

Prevents Path Traversal attacks and avoids filename collisions from identical uploads.

Q17: What if candidate hash doesn't match DB?

AquaMark tests the Ed25519 signature and extracts the DCT watermark. If the payload matches, it confirms authentic content.

Q18: What is YCbCr color space?

Separates Luminance (Y - brightness) from Chrominance (Cb, Cr - color). We embed into Y because human eyes are less sensitive to brightness modulations.

Q19: What is the trade-off of DCT watermarking?

Carrier pages are rendered to 150 DPI raster images to allow pixel DCT modification, slightly increasing file size.

Q20: What are your watermark's limitations?

Severe cropping (>30%) or extreme rotational warping disrupts 8x8 block alignment and prevents recovery.

Q21: How are private signing keys protected?

Stored outside the web server root in backend/storage/keys/ and never exposed to the frontend browser.

Q22: What role-based access is implemented?

First registered user is 'admin', subsequent are 'instructor'. Ownership checks are enforced on every document endpoint.

Q23: How are PDF verification certificates made?

Generated on-the-fly using Python's ReportLab library with full cryptographic hashes and verification verdicts.

Q24: What database is used?

SQLAlchemy ORM with SQLite for zero-config development/tests, swappable to PostgreSQL via DATABASE_URL in docker-compose.yml.

Q25: What is the key takeaway of this project?

Real document security requires defense-in-depth: combining visible watermarks, invisible frequency steganography, and cryptographic signatures.

9. Technical Glossary & Student Revision Checklist

• **2D-DCT:** Two-Dimensional Discrete Cosine Transform. Converts spatial pixel brightness into frequency coefficients.
• **AES-256-GCM:** Advanced Encryption Standard with 256-bit key in Galois/Counter Mode providing confidentiality and integrity.
• **Argon2id:** Hybrid memory-hard password hashing algorithm resistant to GPU cracking.
• **BER:** Bit Error Rate. Percentage of corrupted bits in the extracted payload.
• **Ed25519:** Edwards-curve Digital Signature Algorithm providing 128-bit security with 64-byte signatures.
• **PSNR:** Peak Signal-to-Noise Ratio in dB measuring reconstructed image visual quality.
• **Steganography:** The practice of concealing a secret message within an ordinary carrier file.

Student Master Revision Checklist:
[ ] I can explain the difference between Hashing, Signatures, and Watermarking without mixing them up.
[ ] I know why we modify mid-frequency DCT coefficients (3,2) and (2,3).
[ ] I can demonstrate uploading a PDF, applying an invisible watermark, and verifying it live.
[ ] I understand why a hash mismatch does not automatically prove malicious tampering.
[ ] I know that Ed25519 proves manifest integrity, not original legal copyright ownership.
[ ] I can explain PSNR (visual quality) and Bit Error Rate (payload recovery) in the Robustness Lab.